



山东大学
SHANDONG UNIVERSITY

网络与大数据安全

1 - Overview

李琨

Email: kunli@sdu.edu.cn

—— 学无止境 气有浩然 ——

数字证书

為天下儲人材 為國家圖富強

—— 学无止境 气有浩然 ——



数字证书：证明公钥拥有者身份的凭证

数字证书一般由数字证书认证机构签发，需要

- 申请者通过非对称加密算法 (**RSA**) 生成一对公钥和密钥，然后把需要的申请信息（国家，域名等）连同公钥发送给 证书认证机构 (**CA**)
- **CA** 机构确认无误后通过消息摘要算法 (**MD5**, **SHA**) 生成整个申请信息的摘要签名 **M**，然后把签名 **M** 和使用的摘要算法用 **CA** 自己的私钥进行加密

证书包含了

- 公钥
- 证书拥有者身份信息
- 数字证书认证机构（发行者）信息
- 发行者对这份文件的数字签名及使用的算法
- 有效期



证书认证机构（CA）

数字证书认证机构（英语：**Certificate Authority**，缩写为**CA**），也称为电子商务认证中心、电子商务认证授权机构，是负责发放和管理数字证书的权威机构，并作为电子商务交易中受信任的第三方，承担公钥体系中公钥的合法性检验的责任。

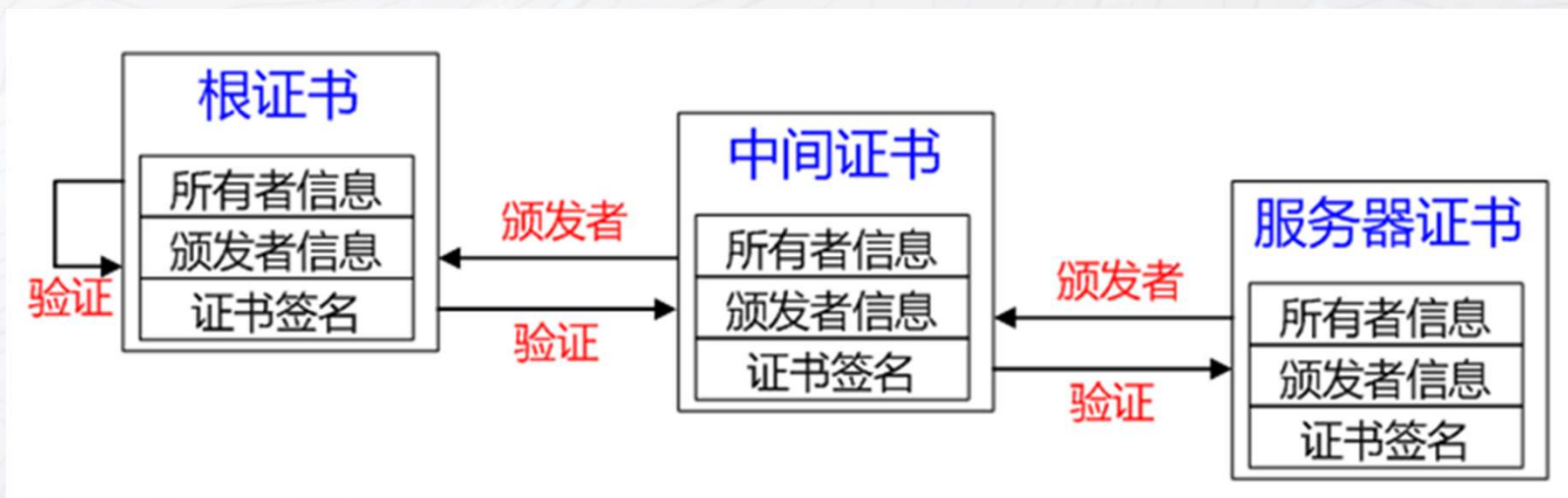
客户端信任这些**CA**，就会在其本地保持这些**CA**的 根证书（**root certificate**），根证书是**CA**自己的证书，是证书验证链的开头。

根证书没有机构（已经是权威了）再为其做数字签名，都是自签证书。

CA会通过 中介证书（**intermediate-certificate**） 替代根证书的去给服务器端的证书签名，确保根证书密钥绝对不可访问。



证书信任链：三级结构

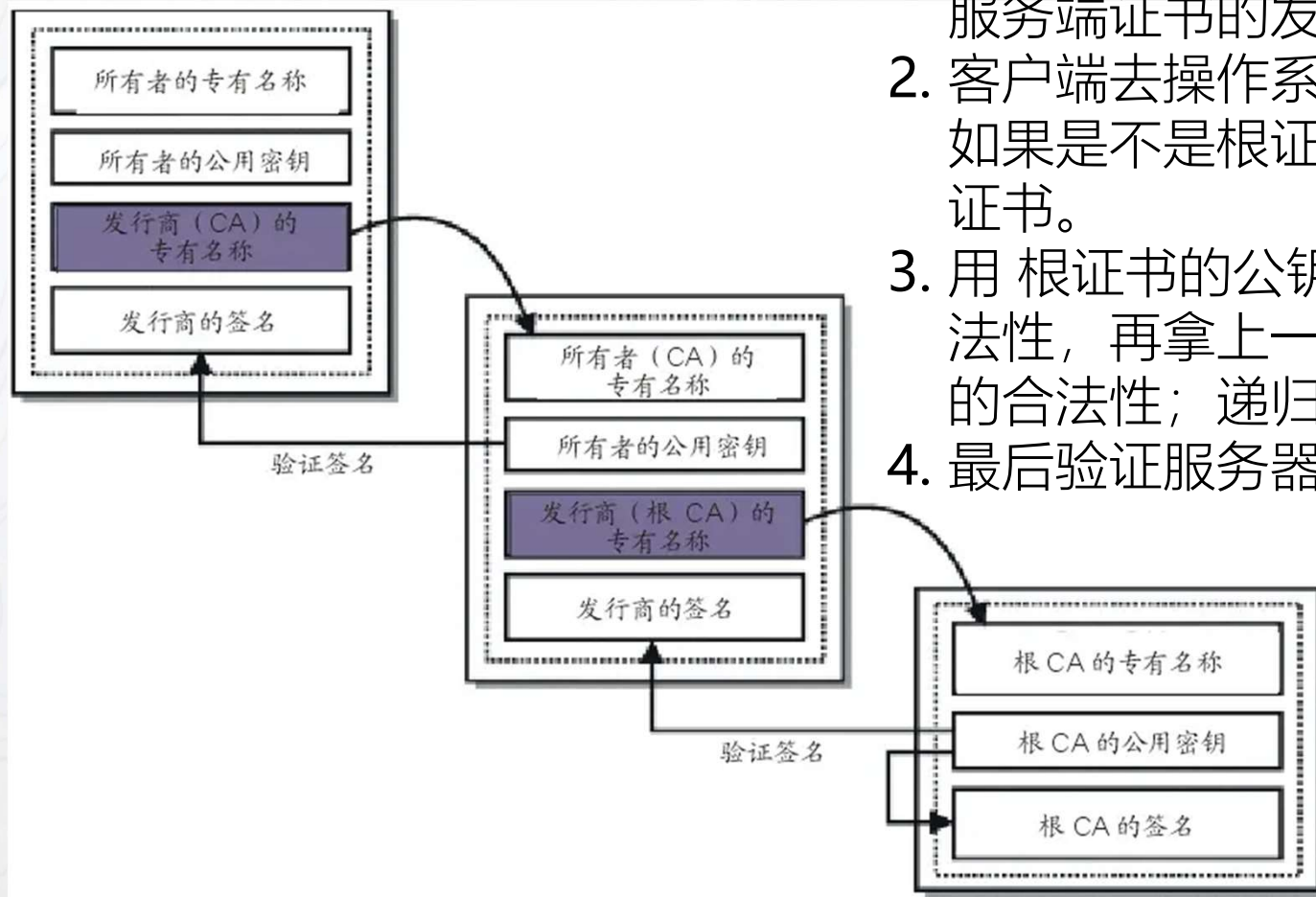


为什么不可以直接都用根证书？

- 安全性问题
- 管理复杂性
- 信任体系崩溃



证书信任链



1. 客户端得到服务端返回的证书，通过读取得到服务端证书的发布机构。
2. 客户端去操作系统查找这个发布机构的的证书，如果不是根证书就继续递归下去 直到拿到根证书。
3. 用 根证书的公钥 去 解密验证 上一层证书的合法性，再拿上一层证书的公钥去验证更上层证书的合法性；递归回溯。
4. 最后验证服务器端的证书是 可信任 的。



自签名证书

自签名证书是指由用户自己生成和签名的证书，而不是由公认的证书颁发机构签名的证书。自签名证书是免费的，但通常不受浏览器和其他客户端的信任。

通常适用于简单的、小规模的环境，或者测试和开发阶段，缺乏集中管理和验证的能力。

优势

- 免费
- 随时签发
- 加密
- 方便

缺陷

- 不受信任
- 不安全

创建流程

- 生成私钥
- 生成CSR（证书签名请求文件）
- 颁发证书



OpenSSL

OpenSSL是一个强大的开源工具，用于实现SSL和TLS协议，保障网络通信的安全。它不仅提供了加密库，还包括了命令行工具，可以用于创建证书、生成密钥、测试SSL/TLS连接等。

主要功能

- 加密算法支持 (Encryption Algorithm Support): 支持多种加密算法，如RSA、AES等。
- SSL/TLS协议实现 (SSL/TLS Protocol Implementation): 提供SSL v2/v3和TLS协议的实现。
- 证书处理 (Certificate Handling): 生成和管理SSL证书。

应用场景

- 安全通信 (Secure Communication): 用于网站、API等的加密通信。
- 数据加密 (Data Encryption): 用于文件、消息等的加密。



OpenSSL下载

1. 下载源码编译（需要提前安装perl和NASM）
2. 使用git bash
3. SSH登录linux
- 4. 下载编译好的版本直接使用**

<https://slproweb.com/products/Win32OpenSSL.html>



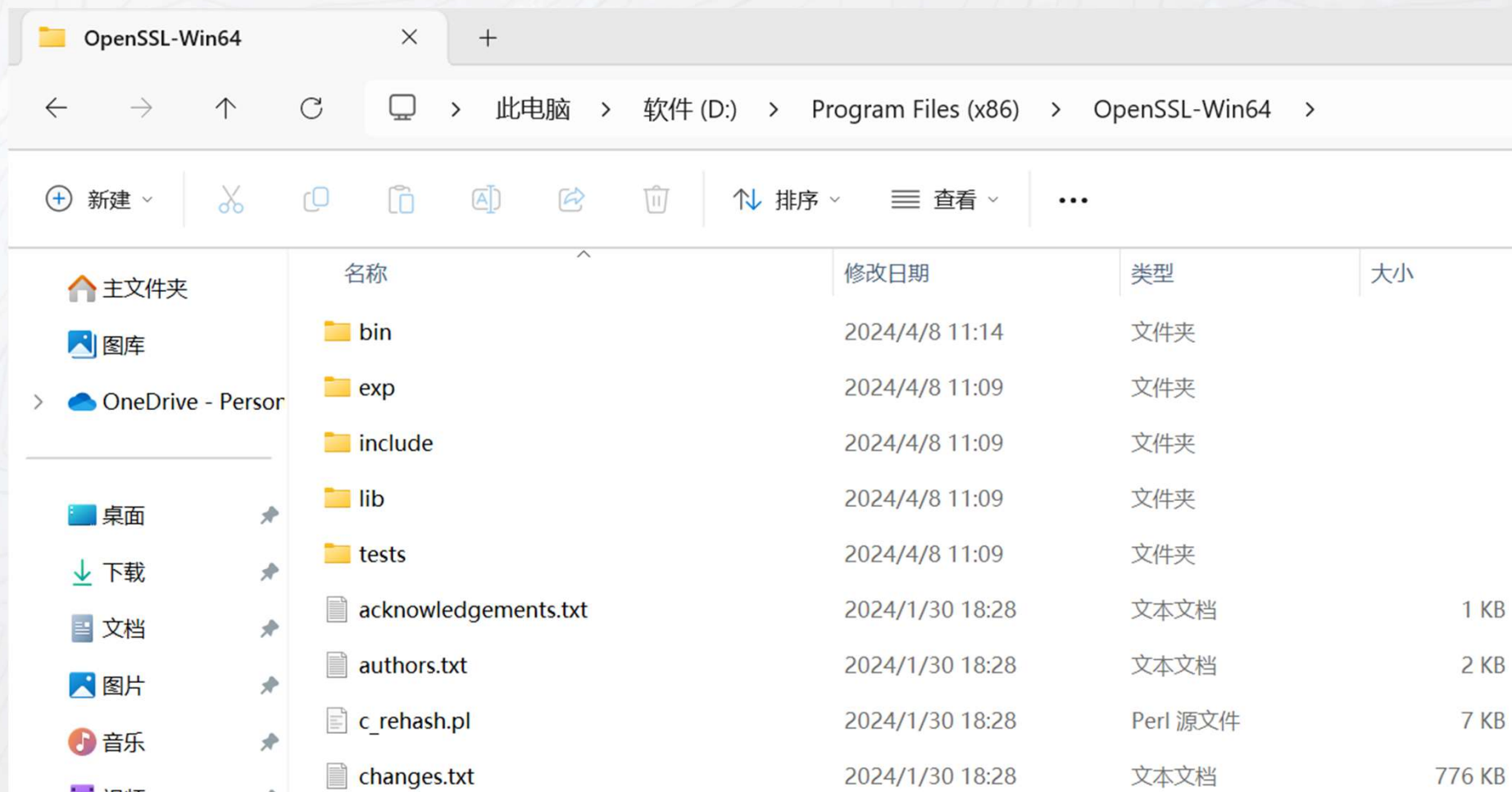
OpenSSL下载 <https://slproweb.com/products/Win32OpenSSL.html>

- 1.
- 2.
- 3.
- 4.

Download Win32/Win64 OpenSSL		
Download Win32/Win64 OpenSSL today using the links below!		
File	Type	Description
Win64 OpenSSL v3.2.1 Light EXE MSI	5MB Installer	Installs the most commonly used essentials of Win64 OpenSSL v3.2.1 (Recommended for users by the creators of OpenSSL). Only installs on 64-bit versions of Windows and targets Intel x64 chipsets. Note that this is a default build of OpenSSL and is subject to local and state laws. More information can be found in the legal agreement of the installation.
Win64 OpenSSL v3.2.1 EXE MSI	200MB Installer	Installs Win64 OpenSSL v3.2.1 (Recommended for software developers by the creators of OpenSSL). Only installs on 64-bit versions of Windows and targets Intel x64 chipsets. Note that this is a default build of OpenSSL and is subject to local and state laws. More information can be found in the legal agreement of the installation.
Win32 OpenSSL v3.2.1 Light EXE MSI	4MB Installer	Installs the most commonly used essentials of Win32 OpenSSL v3.2.1 (Only install this if you need 32-bit OpenSSL for Windows). Note that this is a default build of OpenSSL and is subject to local and state laws. More information can be found in the legal agreement of the installation.
Win32 OpenSSL v3.2.1 EXE MSI	162MB Installer	Installs Win32 OpenSSL v3.2.1 (Only install this if you need 32-bit OpenSSL for Windows). Note that this is a default build of OpenSSL and is subject to local and state laws. More information can be found in the legal agreement of the installation.
Win64 OpenSSL v3.2.1 Light for ARM (EXPERIMENTAL) EXE MSI	6MB Installer	Installs the most commonly used essentials of Win64 OpenSSL v3.2.1 for ARM64 devices (Only install this VERY EXPERIMENTAL build if you want to try 64-bit OpenSSL for Windows on ARM processors). Note that this is a default build of OpenSSL and is subject to local and state laws. More information can be found in the legal agreement of the installation.
Win64 OpenSSL v3.2.1 for ARM (EXPERIMENTAL) EXE MSI	158MB Installer	Installs Win64 OpenSSL v3.2.1 for ARM64 devices (Only install this VERY EXPERIMENTAL build if you want to try 64-bit OpenSSL for Windows on ARM processors). Note that this is a default build of OpenSSL and is subject to local and state laws. More information can be found in the legal agreement of the installation.
Win64 OpenSSL v3.1.5 Light EXE MSI	5MB Installer	Installs the most commonly used essentials of Win64 OpenSSL v3.1.5 (Recommended for users by the creators of OpenSSL). Only installs on 64-bit versions of Windows and targets Intel x64 chipsets. Note that this is a default build of OpenSSL and is subject to local and state laws. More information can be found in the legal agreement of the installation.
Win64 OpenSSL v3.1.5 EXE MSI	142MB Installer	Installs Win64 OpenSSL v3.1.5 (Recommended for software developers by the creators of OpenSSL). Only installs on 64-bit versions of Windows and targets Intel x64 chipsets. Note that this is a default build of OpenSSL and is subject to local and state laws. More information can be found in the legal agreement of the installation.
Win32 OpenSSL v3.1.5 Light EXE MSI	4MB Installer	Installs the most commonly used essentials of Win32 OpenSSL v3.1.5 (Only install this if you need 32-bit OpenSSL for Windows). Note that this is a default build of OpenSSL and is subject to local and state laws. More information can be found in the legal agreement of the installation.
Win32 OpenSSL v3.1.5 EXE MSI	114MB Installer	Installs Win32 OpenSSL v3.1.5 (Only install this if you need 32-bit OpenSSL for Windows). Note that this is a default build of OpenSSL and is subject to local and state laws. More information can be found in the legal agreement of the installation.
Win64 OpenSSL v3.1.5 Light for ARM (EXPERIMENTAL) EXE MSI	5MB Installer	Installs the most commonly used essentials of Win64 OpenSSL v3.1.5 for ARM64 devices (Only install this VERY EXPERIMENTAL build if you want to try 64-bit OpenSSL for Windows on ARM processors). Note that this is a default build of OpenSSL and is subject to local and state laws. More information can be found in the legal agreement of the installation.
Win64 OpenSSL v3.1.5 for ARM (EXPERIMENTAL) EXE MSI	110MB Installer	Installs Win64 OpenSSL v3.1.5 for ARM64 devices (Only install this VERY EXPERIMENTAL build if you want to try 64-bit OpenSSL for Windows on ARM processors). Note that this is a default build of OpenSSL and is subject to local and state laws. More information can be found in the legal agreement of the installation.
Win64 OpenSSL v3.0.13 Light EXE MSI	5MB Installer	Installs the most commonly used essentials of Win64 OpenSSL v3.0.13 (Recommended for users by the creators of OpenSSL). Only installs on 64-bit versions of Windows and targets Intel x64 chipsets. Note that this is a default build of OpenSSL and is subject to local and state laws. More information can be found in the legal agreement of the installation.



OpenSSL使用





OpenSSL使用：添加环境变量

系统属性

计算机名 硬件 高级 系统保护 远程

要进行大多数更改，你必须作为管理员登录。

性能

视觉效果，处理器计划，内存使用，以及虚拟内存

设置(S)...

用户配置文件

与登录帐户相关的桌面设置

设置(E)...

启动和故障恢复

系统启动、系统故障和调试信息

设置(T)...

环境变量(N)...

确定

取消

应用(A)

环境变量

likun 的用户变量(U)

变量	值
OneDrive	C:\Users\likun\OneDrive
Path	C:\Users\likun\AppData\Local\activestate\cache\bin;C:\Users\likun...
TEMP	C:\Users\likun\AppData\Local\Temp
TMP	C:\Users\likun\AppData\Local\Temp

新建(N)...

编辑(E)...

删除(D)

系统变量(S)

变量	值
ComSpec	C:\WINDOWS\system32\cmd.exe
DriverData	C:\Windows\System32\Drivers\DriverData
NUMBER_OF_PROCESSORS	12
OS	Windows_NT
Path	C:\WINDOWS\system32;C:\WINDOWS;C:\WINDOWS\System32\W...
PATHEXT	.COM;.EXE;.BAT;.CMD;.VBS;.VBE;.JS;.JSE;.WSF;.WSH;.MSC
PROCESSOR_ARCHITECTURE	AMD64
PROCESSOR_IDENTIFIER	Intel64 Family 6 Model 151 Stepping 5 GenuineIntel

双击

新建(W)...

编辑(I)...

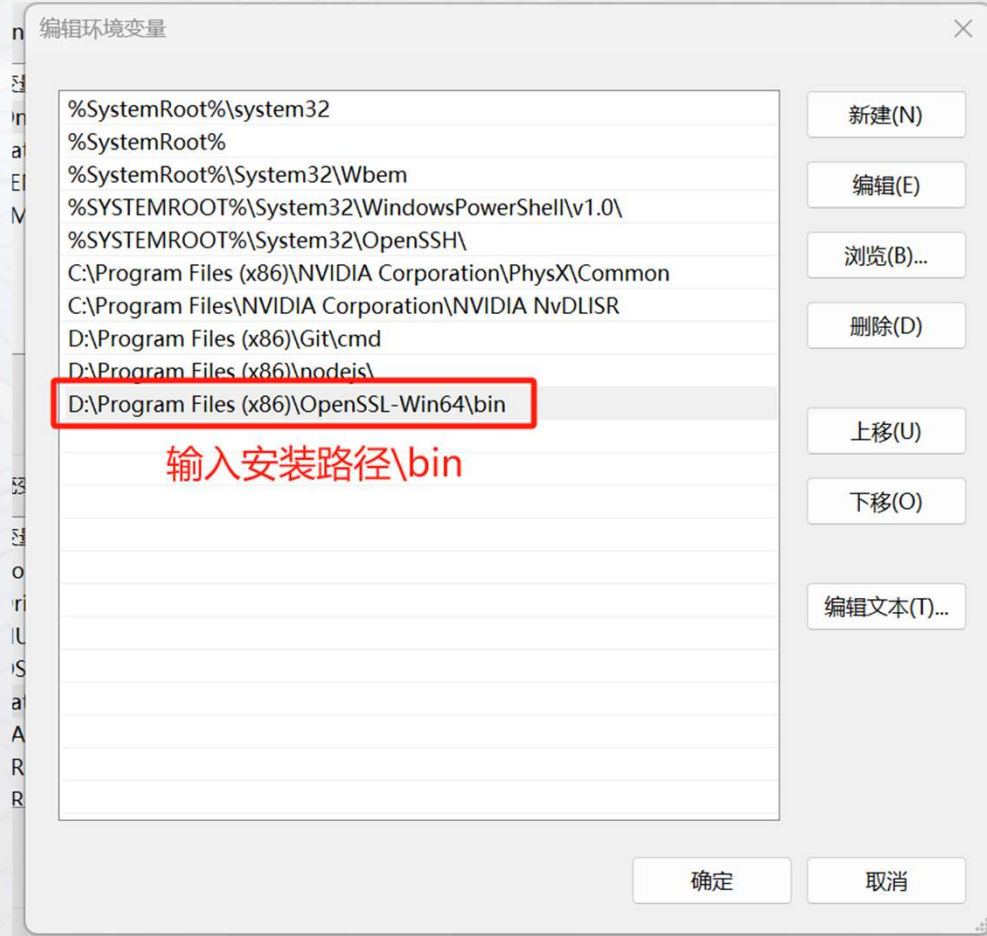
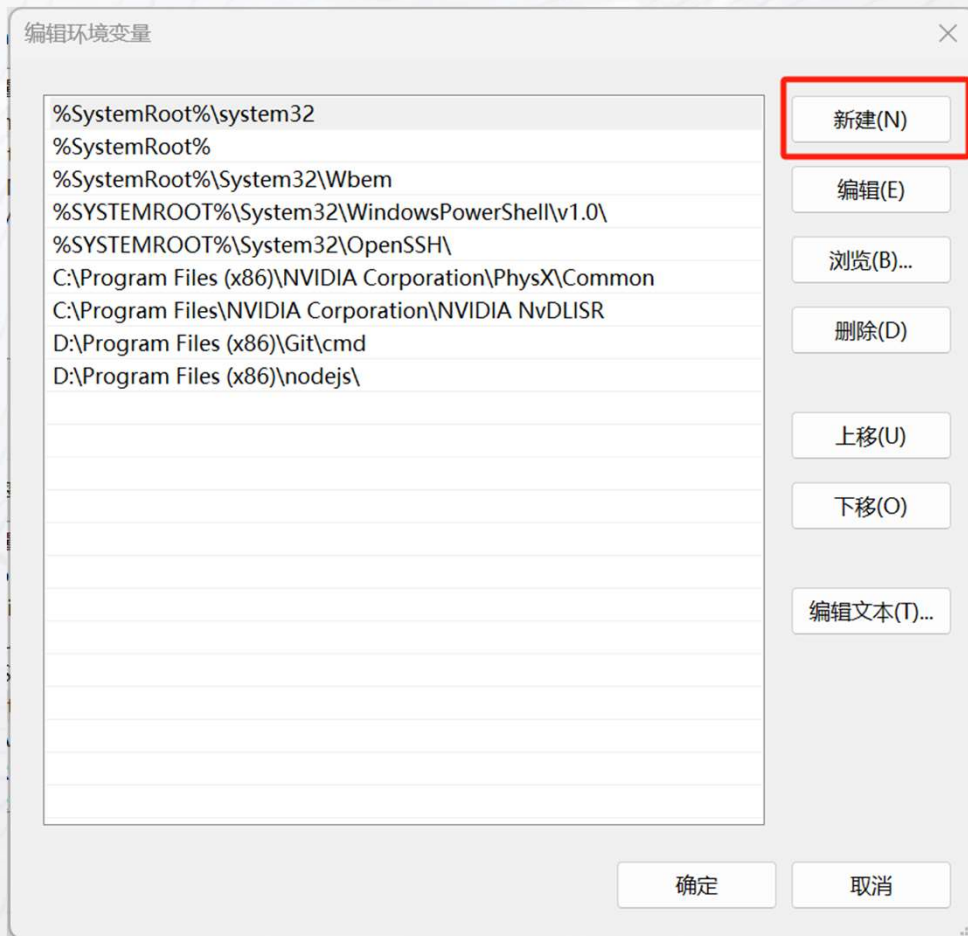
删除(L)

确定

取消



OpenSSL使用：添加环境变量





OpenSSL使用

1. openssl version
2. openssl version -a
3. openssl genrsa -help
4. openssl genrsa -out server.key 2048
5. openssl rsa -in server.key -pubout
6. openssl pkey -in server.key -text

```
C:\WINDOWS\system32\cmd. x + v
D:\>openssl genrsa -help
Usage: genrsa [options] numbits

General options:
  -help          Display this summary
  -engine val    Use engine, possibly a hardware device

Input options:
  -3             (deprecated) Use 3 for the E value
  -F4            Use the Fermat number F4 (0x10001) for the E value
  -f4            Use the Fermat number F4 (0x10001) for the E value

Output options:
  -out outfile   Output the key to specified file
  -passout val   Output file pass phrase source
  -primes +int   Specify number of primes
  -verbose      Verbose output
  -quiet        Terse output
  -traditional  Use traditional format for private keys
  -*            Encrypt the output with any supported cipher

Random state options:
  -rand val      Load the given file(s) into the random number generator
  -writrand outfile Write random data to the specified file

Provider options:
  -provider-path val Provider load path (must be before 'provider' argument if required)
  -provider val    Provider to load (can be specified multiple times)
  -propquery val   Property query used when fetching algorithms
```

```
Windows PowerShell
版权所有 (C) Microsoft Corporation. 保留所有权利。

安装最新的 PowerShell, 了解新功能和改进! https://aka.ms/PSWindows

PS D:\Program Files (x86)\OpenSSL-Win64> openssl
help:

Standard commands
asn1parse      ca             ciphers        cmp
cms            crl            crl2pkcs7      dgst
dhparam        dsa            dsaparam        ec
ecparam        enc            engine          errstr
fipsinstall    gensa          genpkey         genrsa
help           info           kdf             list
mac            nseq           oasp            passwd
pkcs12         pkcs7          pkcs8           pkey
pkeyparam      pkeyutl        prime           rand
rehash         req            rsa             rsautl
s_client       s_server       s_time          sess_id
smime          speed          spkac           srp
storeutl       ts             verify           version
x509

Message Digest commands (see the 'dgst' command for more details)
blake2b512     blake2s256    md4             md5
mdc2           rmd160         sha1            sha224
sha256         sha3-224       sha3-256        sha3-384
sha3-512       sha384         sha512          sha512-224
sha512-256     shake128        shake256         sm3
```

```
C:\WINDOWS\system32\cmd. x + v
D:\>openssl genrsa -out server.key 2048

D:\>openssl rsa -in server.key -pubout
writing RSA key
-----BEGIN PUBLIC KEY-----
MIIBIjANBgkqhkiG9w0BAQEFAAOCAQ8AMIIBCgKCAQEAxnhFda5k2d1m9dfrYUFm
9SV/ugt/Hos3S/XN0PZTFbmmY2+UhcwELVQMcVl3brKh40/rJEYjz3DuSdazX4
db+qObZwldZEwbPk1SP9t4lP+6Yg2T80c1BTFZIE/++Z5COCt0/m6bbijZgtZ0v
qzmFz78H4u++XNSM5qMlt8aGcy1013R26h05e1S7/RvinHqEGECP/bPvVgKvJ
zUYC3nH8W8gYVb2VwSazPyCUhCLruLPR/1egmz33TscDlyS1ui7ZnczvwAHvG
T1iveSR1NcXQNKYPq04Irgty1+uhxoUyNUNCGr5HguStt2+qMlmht/Tqu+bFGH
RwIDAQAB
-----END PUBLIC KEY-----

D:\>openssl pkey -in server.key -text
-----BEGIN PRIVATE KEY-----
MIIEvQIBADANBgkqhkiG9w0BAQEFAASCBKcwggSjAgEAAoIBAQQGeEV0DmTZ3Wb1
1+thQWb1JX+6C38eizdL9c3Q9LMVubBjbSSELASVWIAxy8vdusqHg7+uMRIpNnc0
SINrNfh1v6o5tnDV1kT8umQjk/1PiU/7piDZME5yIFMXNkT/7H5nkI4K1D+bpntuK
NmC1nS+DOYXNPwcrjD5c3kybmoyW3xoYlKI7ULHqHTL4hLv9G+KfioQZEhI/9s+
+FUASgnNRglFefHxbyBgpvZjBJrM/IJSEKwu4s9H/V6CbPfd0xwPXJKW6K7slmdz
Q/AA28ZPXKK971HU21dA2RgrTgiuC3LX66HGHTI100KvkeCSK23b6+oyWaG390
q75sUYdHAgMBAAGCggEAAALt2mkbfem3P8uEzSSJJ+Q4U10U1AH2a4B6A47Vpgso
Ls6TDGmpA0Zjn3SWSL/MgoLXdd+wquNnVN0VPaR0nRbkbDXB+WCI2XGY/YfEpvgN
NZmUr6mG1cVeWlQ74MNgczWnrh1vcpsgIPXlv0sPwmM/fAAjfcEX1NOWXqEbC/qz
6QqKGUEZurCq/tqCwpSdg0hUPVhC5mrQm/1SmCsbmX5Nur7otfb1/eIXv4fITGu
```



制作证书

1. 命令行进入openssl\bin文件夹 `cd D:\Program Files (x86)\OpenSSL-Win64`
2. 设置openssl cnf路径
`set OPENSSL_CONF="D:\Program Files (x86)\OpenSSL-Win64\bin\cnf\openssl.cnf"`
3. 创建子目录
`mkdir certs`
`mkdir private`
4. 生成随机数 `openssl rand -out private/.rand 1000`
5. 生成根证书私钥文件 `openssl genrsa -aes256 -out private/cakey.pem 1024`
该命令含义如下：
 1. `genrsa`: 使用RSA算法产生私钥
 2. `-aes256`: 使用256位密钥的AES算法对私钥进行加密
 3. `-out`: 输出文件的路径
 4. `1024`: 指定私钥长度



制作证书

6. 生成根证书签发申请文件(csr文件)

```
openssl req -new -key private/cakey.pem -out private/ca.csr -subj  
"/C=CN/ST=Shandong/L=Qingdao/O=SDU/OU=SDU/CN=SDU"
```

该命令含义如下：

1. req: 执行证书签发命令
2. -new: 新证书签发请求
3. -key: 指定私钥路径，这个是上一步产生的key文件
4. -out: 输出的csr文件的路径，这个是即将产生的请求文件，需要用这个文件来产生证书
5. -subj: 证书相关的用户信息(subject的缩写)，一些自己定义的相关信息



制作证书 Subject字段说明

字段名	字段值
公用名称 (Common Name)	简称: CN 字段, 对于 SSL 证书, 一般为网站域名; 而对于代码签名证书则为申请单位名称; 而对于客户端证书则为证书申请者的姓名;
单位名称 (Organization Name)	简称: O 字段, 对于 SSL 证书, 一般为网站域名; 而对于代码签名证书则为申请单位名称; 而对于客户端单位证书则为证书申请者所在单位名称;
所在城市 (Locality)	简称: L 字段
所在省份 (State/Province)	简称: S 字段
所在国家 (Country)	简称: C 字段, 只能是国家字母缩写, 如中国: CN
电子邮件 (Email)	简称: E 字段
姓名字段	简称: G 字段
介绍	Description
电话号码	Phone 字段, 格式要求 + 国家区号 城市区号 电话号码, 如: +86 732 88888888
地址	STREET 字段
邮政编码	PostalCode 字段
显示其他内容	简称: OU 字段



制作证书

7. 签发证书

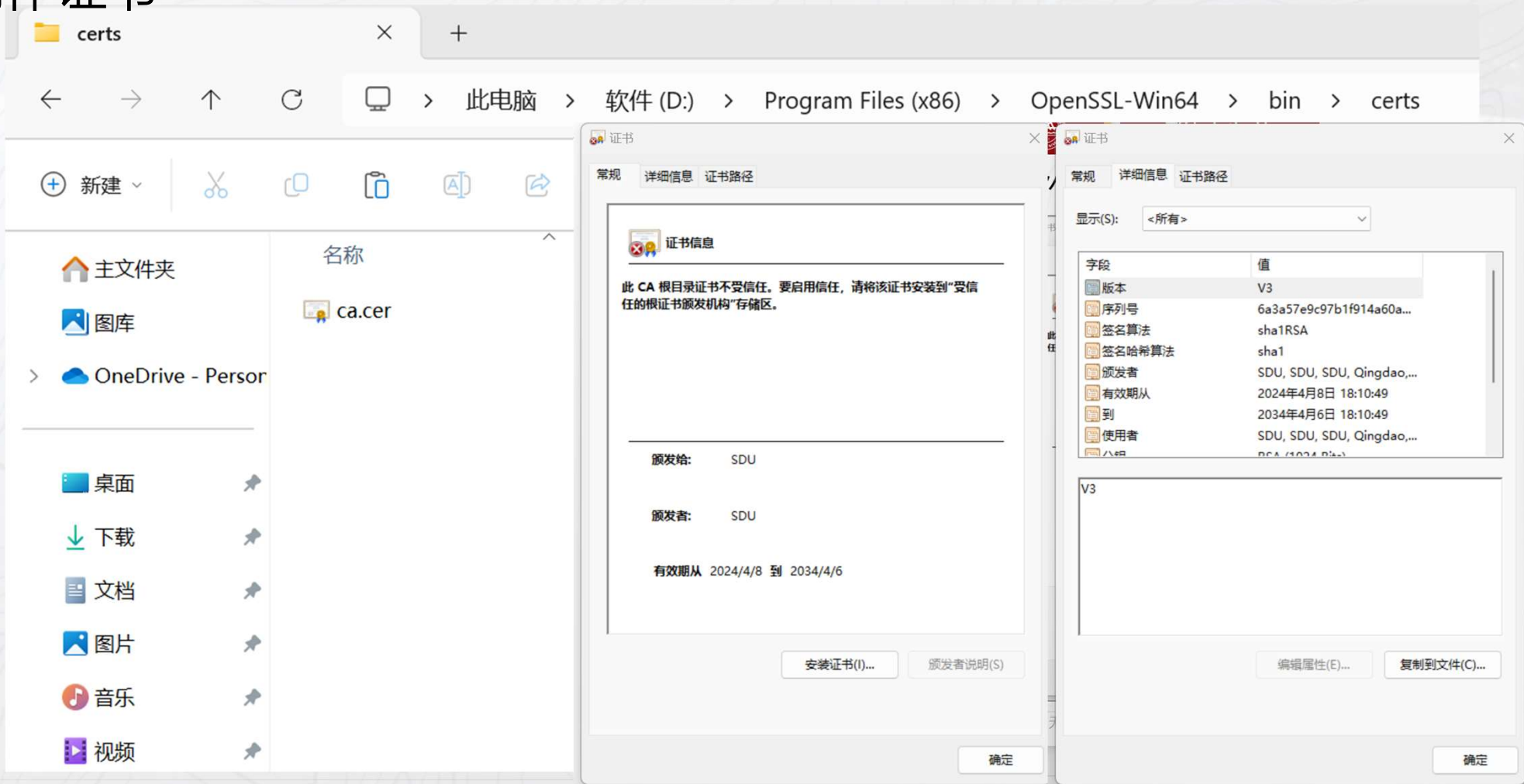
```
openssl x509 -req -days 3650 -sha1 -extensions v3_ca -signkey private/cakey.pem -in private/ca.csr -out certs/ca.cer
```

该命令的相关参数如下：

1. x509: 生成x509格式证书
2. -req: 输入csr文件
3. -days: 证书的有效期（天）
4. -sha1: 证书摘要采用sha1算法
5. -extensions: 按照openssl.cnf文件中配置的v3_ca项添加扩展
6. -signkey: 签发证书的私钥，第一步生成的Key文件
7. -in: 要输入的csr文件,上一步生成的csr文件
8. -out: 输出的cer证书文件



制作证书





服务器证书

1. 生成服务端证书私钥

```
openssl genrsa -aes256 -out private/server.key 1024  
openssl rsa -in private/server.key -out certs/server.key
```

2. 生成请求文件

```
openssl req -new -key private/server.key -out private/server.csr -subj  
"/C=CN/ST=Shandong/L=Qingdao/O=SDU/OU=SDU/CN=SDU"
```

3. 使用根证书签发服务器证书

```
openssl x509 -req -days 7304 -CA certs/ca.cer -CAkey private/cakey.pem -CAserial  
ca.srl -CAcreateserial -in private/server.csr -out certs/server.crt
```

参数解释:

1. CA: 指定CA证书的路径, 为第四步生成的CA证书
2. CAkey: 指定CA证书的私钥路径
3. CAserial: 指定证书序列号文件的路径
4. CAcreateserial: 表示创建证书序列号文件(即上方提到的serial文件), 创建的序列号文件默认名称为-CA, 指定的证书名称后加上.srl后缀



服务器证书





浏览器安装证书

⚙️ 常规

🏠 主页

🔍 搜索 1

🔒 隐私与安全

🔄 同步

Mozilla 产品

安全

欺诈内容和危险软件防护

- ☒ 拦截危险与诈骗内容(B) [详细了解](#)
- ☒ 拦截危险的下载项(D)
- ☒ 发现流氓软件或不寻常的软件时发出警告(C)

证书

- ☒ 查询 OCSP 响应服务器，以确认证书当前是否有效(Q)
- ☒ 允许 Firefox 自动信任您安装的第三方根证书(I) [详细了解](#)

HTTPS-Only 模式

HTTPS 可在 Firefox 和您访问的网站之间提供安全、加密的连接。现今，大多数网站都支持 HTTPS，若选择启用 HTTPS-Only 模式，Firefox 将会升级所有连接为 HTTPS。

[详细了解](#)

- ☐ 在所有窗口启用 HTTPS-Only 模式
- ☐ 仅在隐私窗口启用 HTTPS-Only 模式

2

查看证书...(C)

安全设备...(D)

管理例外...(X)



数字证书

浏览器安装证书

常规 安全

主页 搜索 隐私与安全 同步 Mozilla

证书管理器

您的证书 认证决策 个人 服务器 证书颁发机构

您有用来识别这些证书颁发机构的证书文件

证书名称	安全设备
ACCV	
ACCVRIZ1	Builtin Object Token
Actalis S.p.A./03358520967	
Actalis Authentication Root CA	Builtin Object Token
AffirmTrust	
AffirmTrust Premium ECC	Builtin Object Token
AffirmTrust NetSession	Builtin Object Token

查看...(V) 编辑信任...(E) 导入...(M) 导出...(X) 删除或不信任...(D)

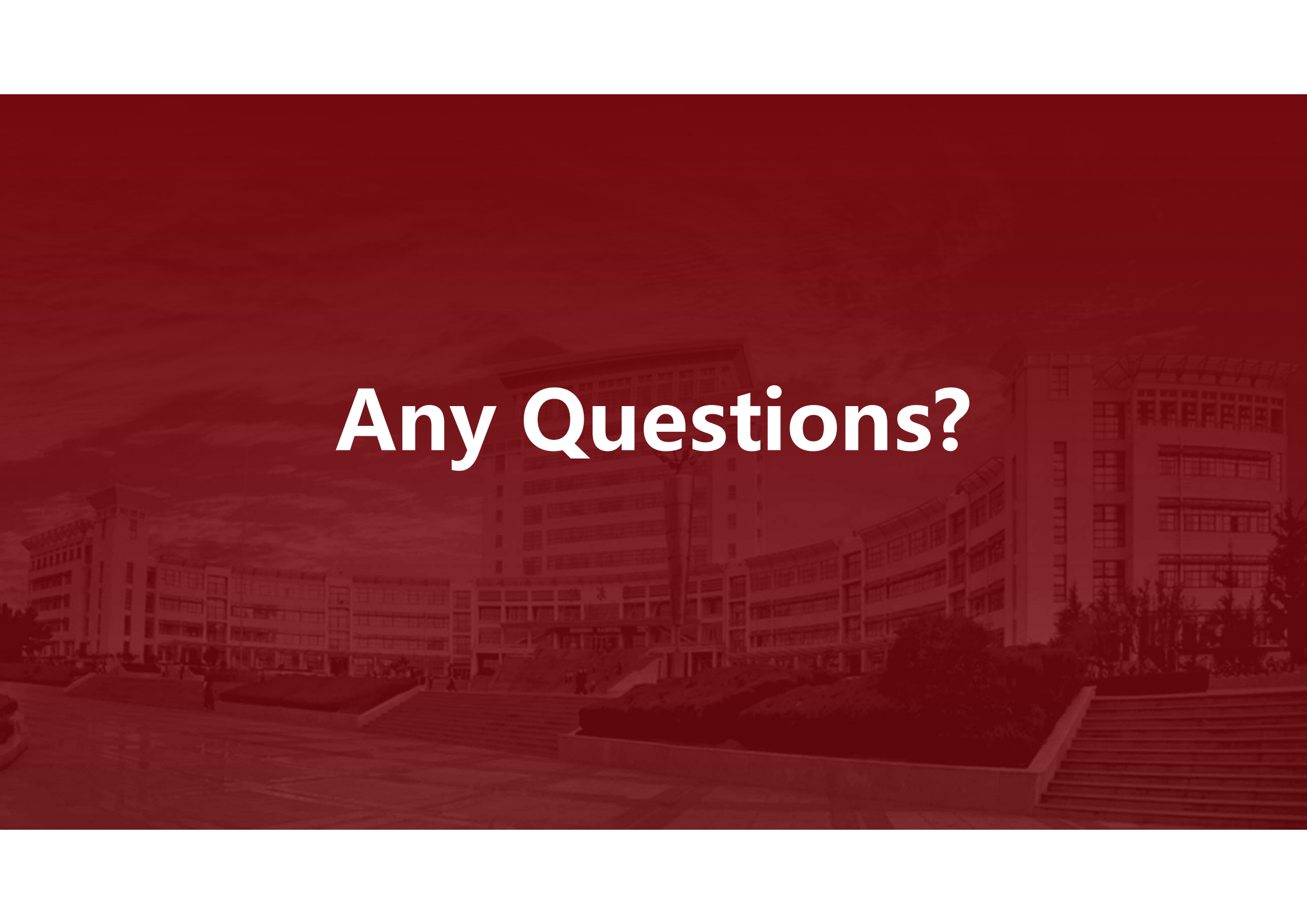
确定

2 查看证书...(C) 安全设备...(D)

多数网站都支持 TPS。

管理例外...(X)

☐ 在所有窗口启用 HTTPS-Only 模式
☐ 仅在隐私窗口启用 HTTPS-Only 模式

A photograph of a university campus with several large, multi-story academic buildings and a central courtyard with stairs and landscaping. The entire image is overlaid with a semi-transparent dark red filter.

Any Questions?

哈希函数

為天下儲人材 為國家圖富強

—— 学无止境 气有浩然 ——



哈希函数Hash

也称为“散列函数”，用来建立目标元素和存储位置（槽位）映射关系的函数
目标元素和存储位置的对应表格，被称为哈希表，存储位置被称为哈希地址

示例：除以7求余数

数据元素	哈希值
2	2
6	6
9	2
25	4
63	0
99	1

对不同的关键字可能得到同一哈希地址，即 $key1 \neq key2$ ，而 $f(key1) = f(key2)$ ，这种现象称碰撞。

具有相同函数值的关键字对该哈希函数来说称做同义词。

若对于关键字集合中的任一个关键字，经哈希函数映射到地址集合中任何一个地址的概率是相等的，则称此类哈希函数为均匀哈希函数(**Uniform Hash function**)，这就是使关键字经过哈希函数得到一个“随机的地址”，从而减少冲突。



哈希函数与质数

对非质数取余时若被除数和除数存在公因子，运算结果的样本空间将缩小，导致哈希碰撞的概率提升。

质数 in Python

```
def find_prime(num):  
    if num > 1: # 查看因子  
        for i in range(2, num):  
            if (num % i) == 0:  
                print(num, "不是质数")  
                print(i, "乘以", num//i, "是", num)  
                break  
        else:  
            print(num, "是质数")  
    else:  
        print(num, "不是质数") # 如果输入的数字小于或等于 1, 不是
```

```
def find_prime_2(num):  
    primes_bool = [False, False] + [True] * (num - 1)  
    for i in range(3, len(primes_bool)):  
        if i & 1 == 0:  
            primes_bool[i] = False  
        for i in range(3, int(math.sqrt(num)) + 1):  
            if primes_bool[i] is True:  
                for j in range(i + i, num + 1, i):  
                    primes_bool[j] = False  
    primes = []  
    [primes.append(i) for i, v in  
     enumerate(primes_bool) if v is True]  
    return primes
```



哈希表 in Python

```
class hashtable(object):
    def __init__(self):
        self.items=[]
    def put(self,k,v):
        #添加对应关系
        self.items.append((k,v))
    def get(self,k):
        for key,value in self.items:
            if(k==key):
                #返回正确匹配出的结果
                return value
```

```
class hashtable(object):
    def __init__(self):
        self.items=[None]*100
    def hash(self,a):
        return a*1+1 #线性映射关系
    def put(self,k,v):
        #根据哈希结果, 添加映射关系
        self.items[self.hash(k)]=v
    def get(self,k):
        hashcode=self.hash(k)
        #根据哈希结果, 返回正确匹配出结果
        return self.items[hashcode]
```



处理冲突：开放寻址法

$H_i = (H(key) + d_i) \text{ MOD } m, i = 1, 2, \dots, k (k \leq m - 1)$, 其中 $H(key)$ 为散列函数, m 为散列表长, d_i 为增量序列,

$$hash_i = (hash(key) + d_i) \text{ mod } m, i = 1, 2, \dots, k (k \leq m - 1)$$

d_i 有下列三种取法:

1. $d_i = 1, 2, 3, \dots, m - 1$, 称线性探测再散列;

2. $d_i = 1^2, -1^2, 2^2, -2^2, \dots \pm (k)^2, (k \leq \frac{m}{2})$ 称平方探测再散列;

3. $d_i = hash_2(key)$, 称双散列

练习：将关键字序列(7、8、30、11、18、9、14)散列存储到散列表中。散列表的存储空间是一个下标从0开始的一维数组，散列函数为 $H(key) = (key \times 3) \text{ mod } 7$ ，处理冲突采用线性探测再散列法，要求哈希表长度为10。请画出所构造的散列表。



处理冲突：开放寻址法

练习：将关键字序列(7、8、30、11、18、9、14)散列存储到散列表中。散列表的存储空间是一个下标从0开始的一维数组，散列函数为 $H(key) = (key \times 3) \bmod 7$ ，处理冲突采用线性探测再散列法，要求哈希表长度为10。请画出所构造的散列表。

key	7	8	30	11	18	9	14
hash	0	3	6	5	5	6	0

冲突组合：7和14，30和9，11和18

地址	0	1	2	3	4	5	6	7	8	9
关键字	7									



处理冲突：开放寻址法

练习：将关键字序列(7、8、30、11、18、9、14)散列存储到散列表中。散列表的存储空间是一个下标从0开始的一维数组，散列函数为 $H(key) = (key \times 3) \bmod 7$ ，处理冲突采用线性探测再散列法，要求哈希表长度为10。请画出所构造的散列表。

key	7	8	30	11	18	9	14
hash	0	3	6	5	5	6	0

冲突组合：7和14，30和9，11和18

地址	0	1	2	3	4	5	6	7	8	9
关键字	7			8						



处理冲突：开放寻址法

练习：将关键字序列(7、8、30、11、18、9、14)散列存储到散列表中。散列表的存储空间是一个下标从0开始的一维数组，散列函数为 $H(key) = (key \times 3) \bmod 7$ ，处理冲突采用线性探测再散列法，要求哈希表长度为10。请画出所构造的散列表。

key	7	8	30	11	18	9	14
hash	0	3	6	5	5	6	0

冲突组合：7和14，30和9，11和18

地址	0	1	2	3	4	5	6	7	8	9
关键字	7			8			30			



处理冲突：开放寻址法

练习：将关键字序列(7、8、30、11、18、9、14)散列存储到散列表中。散列表的存储空间是一个下标从0开始的一维数组，散列函数为 $H(key) = (key \times 3) \bmod 7$ ，处理冲突采用线性探测再散列法，要求哈希表长度为10。请画出所构造的散列表。

key	7	8	30	11	18	9	14
hash	0	3	6	5	5	6	0

冲突组合：7和14，30和9，11和18

地址	0	1	2	3	4	5	6	7	8	9
关键字	7			8		11	30			



处理冲突：开放寻址法

练习：将关键字序列(7、8、30、11、18、9、14)散列存储到散列表中。散列表的存储空间是一个下标从0开始的一维数组，散列函数为 $H(key) = (key \times 3) \bmod 7$ ，处理冲突采用线性探测再散列法，要求哈希表长度为10。请画出所构造的散列表。

key	7	8	30	11	18	9	14
hash	0	3	6	5	5	6	0

冲突组合：7和14，30和9，11和18

地址	0	1	2	3	4	5	6	7	8	9
关键字	7			8		11	30	18		



处理冲突：开放寻址法

练习：将关键字序列(7、8、30、11、18、9、14)散列存储到散列表中。散列表的存储空间是一个下标从0开始的一维数组，散列函数为 $H(key) = (key \times 3) \bmod 7$ ，处理冲突采用线性探测再散列法，要求哈希表长度为10。请画出所构造的散列表。

key	7	8	30	11	18	9	14
hash	0	3	6	5	5	6	0

冲突组合：7和14，30和9，11和18

地址	0	1	2	3	4	5	6	7	8	9
关键字	7			8		11	30	18	9	



处理冲突：开放寻址法

练习：将关键字序列(7、8、30、11、18、9、14)散列存储到散列表中。散列表的存储空间是一个下标从0开始的一维数组，散列函数为 $H(key) = (key \times 3) \bmod 7$ ，处理冲突采用线性探测再散列法，要求哈希表长度为10。请画出所构造的散列表。

key	7	8	30	11	18	9	14
hash	0	3	6	5	5	6	0

冲突组合：7和14，30和9，11和18

地址	0	1	2	3	4	5	6	7	8	9
关键字	7	14		8		11	30	18	9	



处理冲突：开放寻址法

练习：将关键字序列(32、36、16、46、71、42、24、49、75、88、51)散列存储到散列表中。散列表的存储空间是一个下标从0开始的一维数组，散列函数为 $H(key) = key \bmod 13$ ，处理冲突采用线性探测再散列法，要求哈希表长度为16。请画出所构造的散列表。



处理冲突：开放寻址法

练习：将关键字序列(32、36、16、46、71、42、24、49、75、88、51)散列存储到散列表中。散列表的存储空间是一个下标从0开始的一维数组，散列函数为 $H(key) = key \bmod 13$ ，处理冲突采用线性探测再散列法，要求哈希表长度为16。请画出所构造的散列表。

地址	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
关键字				16	42		32	46	71		36	24	49	75	88	51



处理冲突：开放寻址法

```
class hashtable(object):  
    def __init__(self):  
        self.hash_table=[[None,None] for i in range(10)]  
    def hash(self,k,i):  
        h_value=(k+i)%10  
        if self.hash_table[h_value][0]==k:  
            return h_value  
        if self.hash_table[h_value][0]!=None:  
            i=i+1  
            h_value=self.hash(k,i)  
        return h_value  
    def put(self,k,v):  
        hash_v=self.hash(k,0)  
        self.hash_table[hash_v][0]=k  
        self.hash_table[hash_v][1]=v  
    def get(self,k):  
        hash_v=self.hash(k,0)  
        return self.hash_table[hash_v][1]
```



处理冲突：开放寻址法——填满了怎么办？

载荷因子 (load factor)

α = 已有的元素个数 / 表的长度 由于表长是定值， α 与“填入表中的元素个数”成正比，所以， α 越大，表明填入表中的元素越多，产生冲突的可能性就越大；反之， α 越小，表明填入表中的元素越少，产生冲突的可能性就越小。

当哈希表填充到达一定程度，通过修改载荷因子来扩容，一般选择扩到已插入元素数量的两倍。



哈希函数

```
class hashtable(object):
    def __init__(self):
        self.capacity=10
        self.hash_table=[[None,None]for i in range(self.capacity)]
        self.num=0
        self.load_factor=0.75
    def hash(self,k,i):...
    def resize(self):
        self.capacity=self.num*2 #扩容到原有元素数量的两倍
        temp=self.hash_table[:]
        self.hash_table=[[None,None]for i in range(self.capacity)]
        for i in temp:
            if(i[0]!=None): #把原来已有的元素存入
                hash_v=self.hash(i[0],0)
                self.hash_table[hash_v][0]=i[0]
                self.hash_table[hash_v][1]=i[1]
    def put(self,k,v):
        ...
        self.num=self.num+1 #暂不考虑key重复的情况，具体自己可以优化
        if(self.num/len(self.hash_table)>self.load_factor):# 如果比例大于载荷因子
            self.resize()
    def get(self,k):...
```



处理冲突：开放寻址法

练习：将关键字序列(47、7、29、11、9、84、54、20、30)散列存储到散列表中。散列表的存储空间是一个下标从0开始的一维数组，散列函数为 $H(key) = key \bmod 11$ ，处理冲突采用平方探测再散列法，要求哈希表长度为11。请画出所构造的散列表。

key	47	7	29	11	9	84	54	20	30
hash	3	7	7	0	9	7	10	9	8

平方探测： $d_i = 1^2, -1^2, 2^2, -2^2, \dots \pm (k)^2$



哈希函数

处理冲突：开放寻址法

key	47	7	29	11	9	84	54	20	30
hash	3	7	7	0	9	7	10	9	8

	0	1	2	3	4	5	6	7	8	9	10	说明
47				47								无冲突
7				47				7				无冲突
29				47				7	29			冲突1次, $d_i = 1$
11	11			47				7	29			无冲突
9	11			47				7	29	9		无冲突
84	11			47			84	7	29	9		冲突2次, $d_i = -1$
54	11			47			84	7	29	9	54	无冲突
20	11		20	47			84	7	29	9	54	冲突3次, $d_i = 2^2 = 4$
30	11	30	20	47			84	7	29	9	54	冲突3次, $d_i = 2^2 = 4$



处理冲突：开放寻址法

练习：将关键字序列(19、1、23、14、55、68、11、82、36)散列存储到散列表中。散列表的存储空间是一个下标从0开始的一维数组，散列函数为 $H(key) = key \bmod 11$ ，处理冲突分别采用线性探测再散列法和平方探测再散列法，要求哈希表长度为11。请画出所构造的散列表。



处理冲突：开放寻址法

练习：将关键字序列(19、1、23、14、55、68、11、82、36)散列存储到散列表中。散列表的存储空间是一个下标从0开始的一维数组，散列函数为 $H(key) = key \bmod 11$ ，处理冲突分别采用线性探测再散列法和平方探测再散列法，要求哈希表长度为11。请画出所构造的散列表。

地址	0	1	2	3	4	5	6	7	8	9	10
关键字	55	1	23	14	68	11	82	36	19		

地址	0	1	2	3	4	5	6	7	8	9	10
关键字	55	1	23	14	36	82	68		19		11



哈希表的关键：哈希算法

哈希算法是一种将任意长度的输入数据转换为固定长度输出的算法。它通过哈希函数对输入数据进行处理，生成称为哈希值或摘要的固定长度输出。

哈希函数的特性

- **确定性**：对于相同的输入，哈希函数始终产生相同的哈希值。
- **快速计算**：哈希函数能够高效地计算出哈希值。
- **不可逆性**：从哈希值无法推导出原始输入数据。
- **雪崩效应**：输入数据的微小改变会导致输出哈希值的巨大变化。
- **抗碰撞性**：很难找到两个不同的输入值，它们经过哈希处理后得到相同的输出值。





常用哈希算法

算法	输出长度（位）	输出长度（字节）
MD5	128 bits	16 bytes
SHA-1	160 bits	20 bytes
RipeMD-160	160 bits	20 bytes
SHA-256	256 bits	32 bytes
SHA-512	512 bits	64 bytes

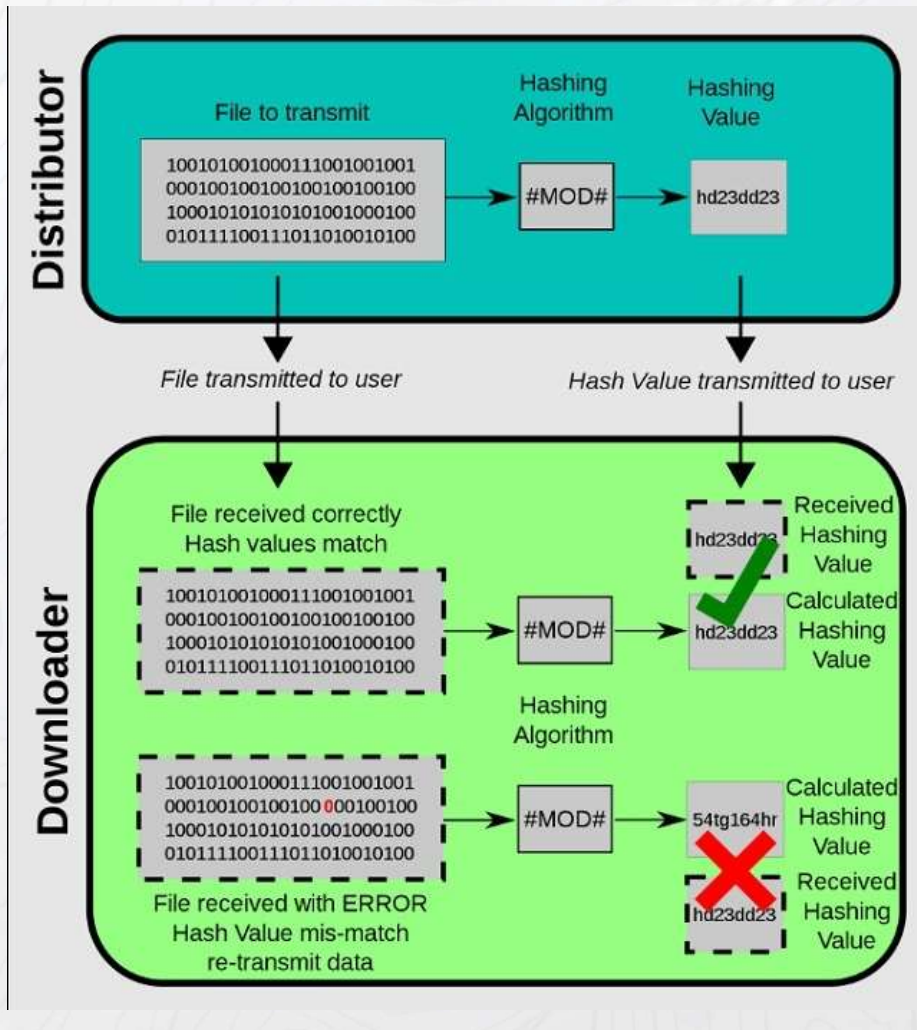
哈希算法的应用

- 数据索引和查找
- 数据完整性验证——哈希校验
- 密码存储（**并不安全**）



哈希函数

哈希校验



对比两个文件的哈希值是否相同来判断文件是否被篡改



哈希校验

对比两个文件的哈希值是否相同来判断文件是否被篡改





哈希校验 in Python

```
import hashlib

# 数据完整性验证
data = "Hello, World!"
original_hash = hashlib.sha256(data.encode()).hexdigest()

# 在传输过程中数据被篡改
modified_data = "Hello, Modified!"
modified_hash = hashlib.sha256(modified_data.encode()).hexdigest()

# 验证数据完整性
if original_hash == modified_hash:
    print("数据完整性验证通过")
else:
    print("数据完整性验证失败")
```



哈希校验

命令行校验: `certutil -hashfile filename SHA256`

```
Windows PowerShell
版权所有 (C) Microsoft Corporation。保留所有权利。

安装最新的 PowerShell, 了解新功能和改进! https://aka.ms/PSWindows

PS D:\Code\python> certutil -hashfile hash_test.txt SHA256
SHA256 的 hash_test.txt 哈希:
05a17114410b01aaefead38464050aa4be82ff685b17b91206fb631915b4646b
CertUtil: -hashfile 命令成功完成。
```

```
27 filename = "hash_test.txt"
28 with open(filename, 'rb') as f:
29     byte = f.read()
30     filehash = hashlib.sha256(byte).hexdigest()
31     print(filehash)
```

问题 输出 调试控制台 终端 端口

```
PS D:\Code\python> d:; cd 'd:\Code\python'; & 'D:\Program Files (x86)\Python\Python311\python.exe' 'c:\Users\likun\.vscode\extensions\ms-python.python-2024.2.1\pythonFiles\lib\python\debugpy\adapter\..\..\debugpy\launcher' '58071' '--' 'D:\Code\python\a.py'
05a17114410b01aaefead38464050aa4be82ff685b17b91206fb631915b4646b
PS D:\Code\python> █
```



哈希存储密码

明文存储：一旦数据库泄露密码会直接暴露

存储密码的哈希：数据库泄露不会直接暴露密码，但可以通过哈希碰撞等方式暴力破解（彩虹表攻击）

username	password	password_hash
bob	hello123	f30aa7a662c728b7407c54ae6bfd27d1
alice	12345678	25d55ad283aa400af464c76d713c07ad
tim	passw0rd	bed128365216c019988915ed3add75fb

密文:

类型: ▼ [\[帮助\]](#)

查询结果:

passw0rd

<https://cmd5.com/>



哈希存储密码

明文存储：一旦数据库泄露密码会直接暴露

存储密码的哈希：数据库泄露不会直接暴露密码，但可以通过哈希碰撞等方式暴力破解（彩虹表攻击）

常用口令	MD5
hello123	f30aa7a662c728b7407c54ae6bfd27d1
12345678	25d55ad283aa400af464c76d713c07ad
passw0rd	bed128365216c019988915ed3add75fb
19700101	570da6d5277a646f6552b8832012f5dc
...	...
20201231	6879c0ae9117b50074ce0a0d4c843060



哈希存储密码

彩虹表攻击防御方式：加盐（salt），对每个密码额外添加随机数

原始数值	salt	32位的MD5值	解密结果
yschlj	无	9ef955becb90f111522b599b91393118	yschlj
yschlj	wka	53dc0f081b142eec156aa32ff568c2ba	yschljwka

要避免：盐值重复、盐值过短、用户名作为盐值

username	salt	password
bob	H1r0a	a5022319ff4c56955e22a74abcc2c210
alice	7\$p2w	e5de688c99e961ed6e560b972dab8b6a
tim	z5Sk9	1eee304b92dc0d105904e7ab58fd2f64



哈希存储密码 in Python

```
import hashlib

# 密码存储
password = "mypassword123"
hashed_password = hashlib.sha256(password.encode()).hexdigest()

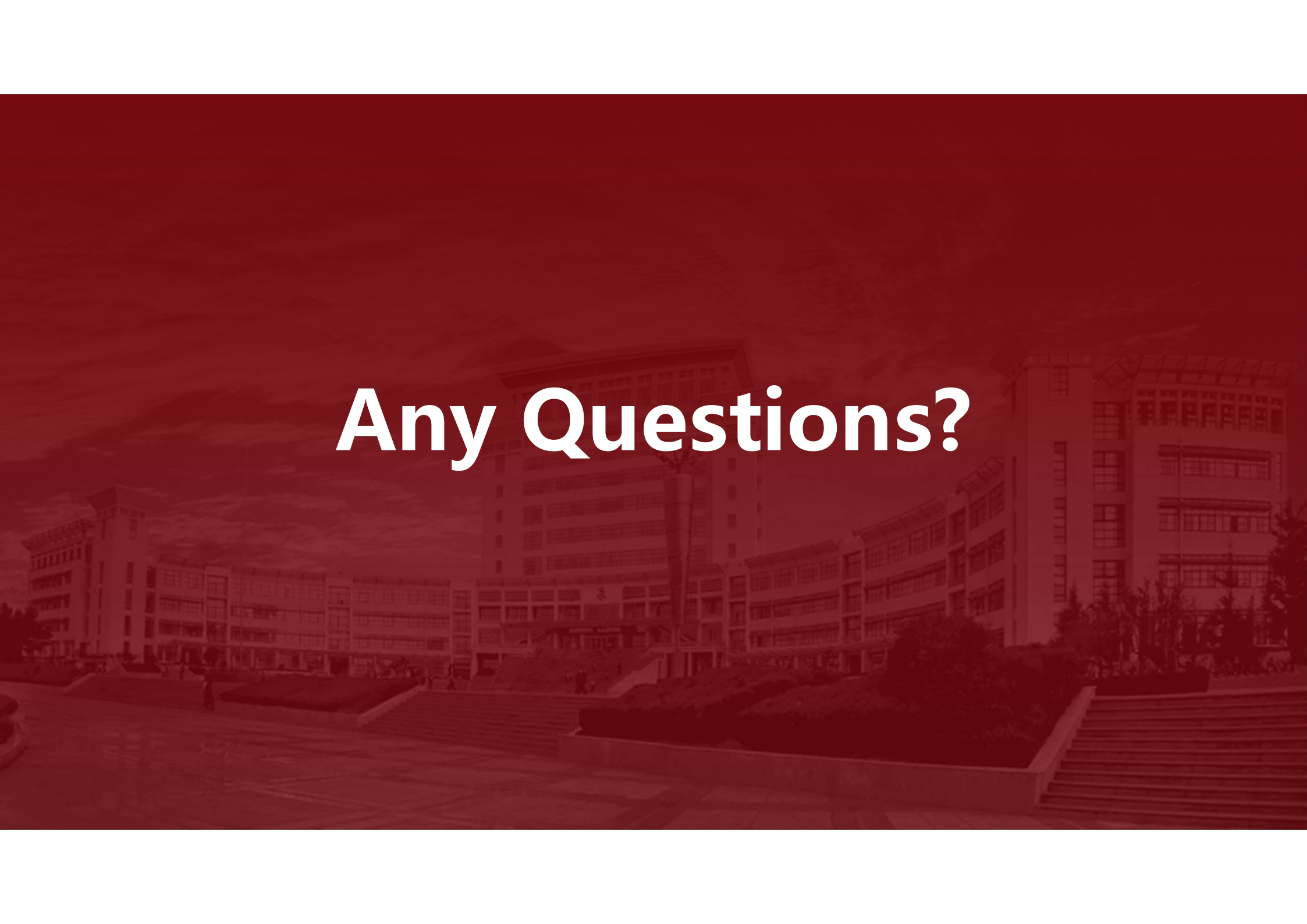
# 假设用户输入密码进行登录
login_password = input("请输入密码: ")
hashed_login_password = hashlib.sha256(login_password.encode()).hexdigest()

# 验证密码
if hashed_password == hashed_login_password:
    print("密码正确, 登录成功")
else:
    print("密码错误, 登录失败")
```



哈希存储密码 in Python

```
import hashlib
# 密码存储
import os
# 生成随机盐
salt = os.urandom(16).hex()
password = "mypassword123"
salted_password = salt + password
hashed_salt_password = hashlib.sha256(salted_password.encode()).hexdigest()
# 假设用户输入密码进行登录
login_password = input("请输入密码: ")
salt_login_password = salt + login_password
hashed_salt_login_password = hashlib.sha256(salt_login_password.encode()).hexdigest()
# 验证密码
if hashed_salt_password == hashed_salt_login_password:
    print("密码正确, 登录成功")
else:
    print("密码错误, 登录失败")
```

A photograph of a university campus with several large, multi-story academic buildings and a central courtyard with stairs and landscaping. The image is overlaid with a semi-transparent dark red filter.

Any Questions?

凯撒密码

為天下儲人材 為國家圖富強

—— 学无止境 气有浩然 ——



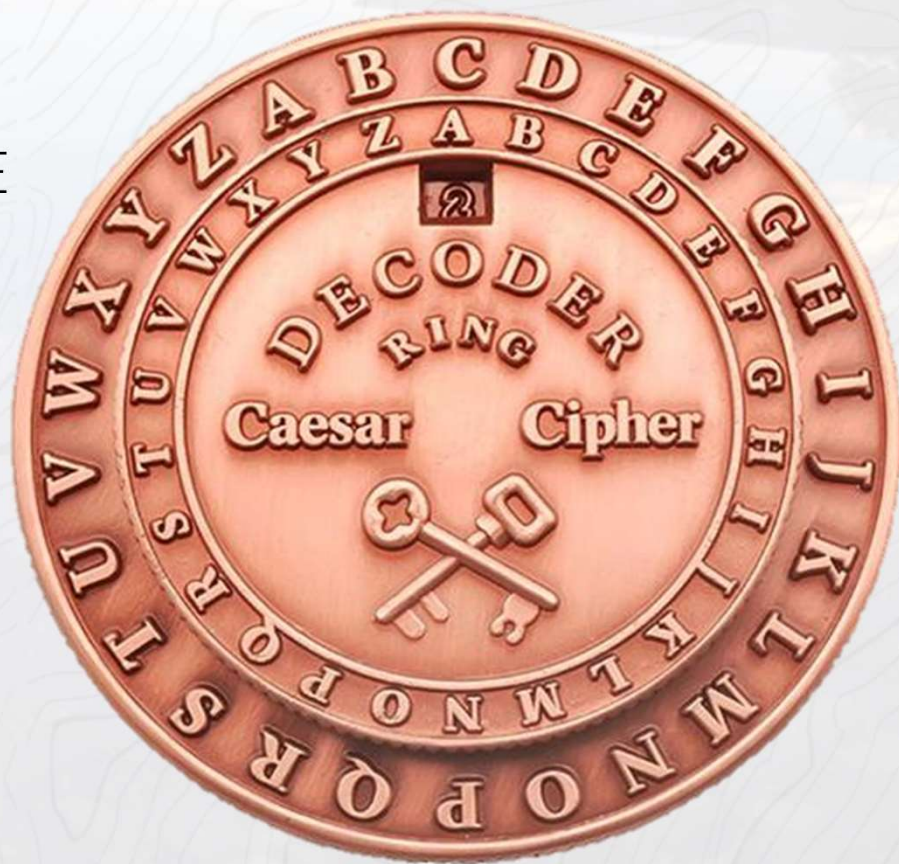
凯撒密码

明文中的所有字母都在字母表上向后（或向前）按照一个固定数目进行偏移后被替换成密文

最早由古罗马军事统帅盖乌斯·尤利乌斯·凯撒在军队中用来传递加密信息

如果需要保密，信中便用暗号，也即是改变字母顺序，使局外人无法组成一个单词。如果想要读懂和理解它们的意思，得用第4个字母置换第一个字母，即以D代A，余此类推。

——苏维托尼乌斯，罗马十二帝王传



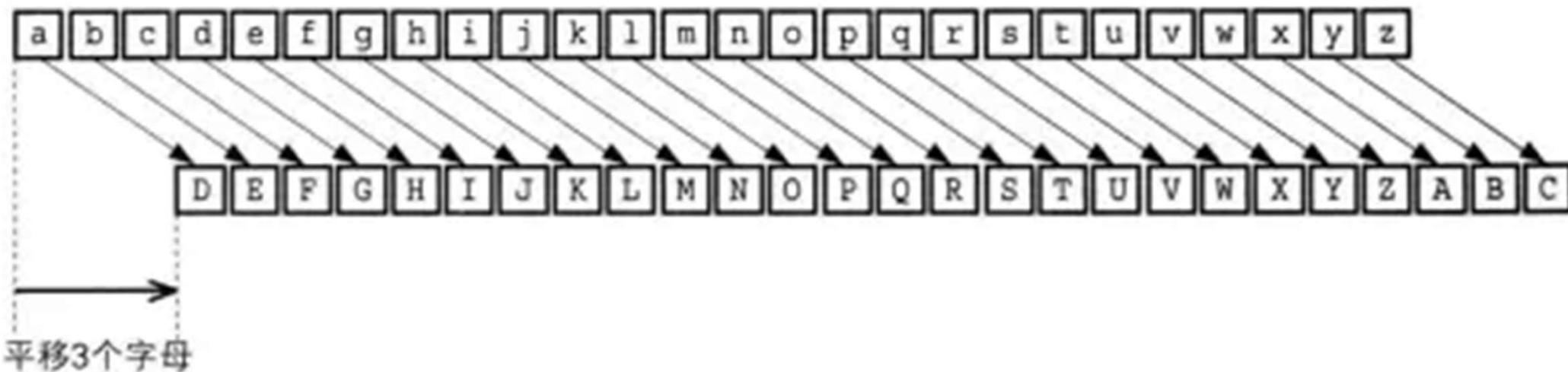
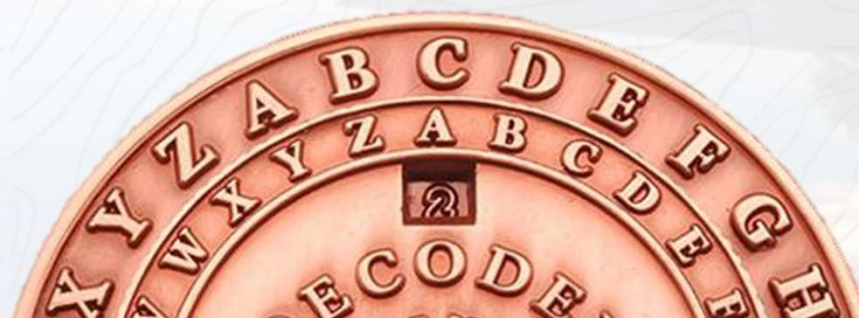


凯撒密码

凯撒密码

明文中的所有字母都在字母表上向后（或向前）按照一个固定数目进行偏移后被替换成密文

最早由古罗马军事统帅盖乌斯·尤利乌斯·凯撒在军队中用来传递加密信息





凯撒密码 in Python

ASCII码

- A~Z对应十进制ASCII编码：65~90
- a~z对应十进制ASCII编码：97~122
- 1~9对应十进制ASCII编码：48~57

chr()函数

用一个范围在range(256)内的整数作参数，返回当前整数对应的ASCII字符，返回值为字符串形式。

ord()函数

输入ASCII字符表中字符的字符串形式，返回其在字符表中的排序位次。



凯撒密码 in Python

```
def caesar_encrypt(plain_text, shift):
    cipher_text=''
    for char in plain_text:
        if char.isalpha():
            ascii_offset=65 if char.isupper() else 97
            cipher_text += chr((ord(char)- ascii_offset + shift) % 26 + ascii_offset)
        else:
            cipher_text += char
    return cipher_text
def caesar_decrypt(cipher_text, shift):
    return caesar_encrypt(cipher_text,-shift)
# 测试
plain_text = "hello world"
shift = 3
encrypt_text = caesar_encrypt(plain_text,shift)
print("加密后的文本是: ", encrypt_text)
decrypted_text = caesar_decrypt(encrypt_text,shift)
print("解密后的文本是: ", decrypted_text)
```



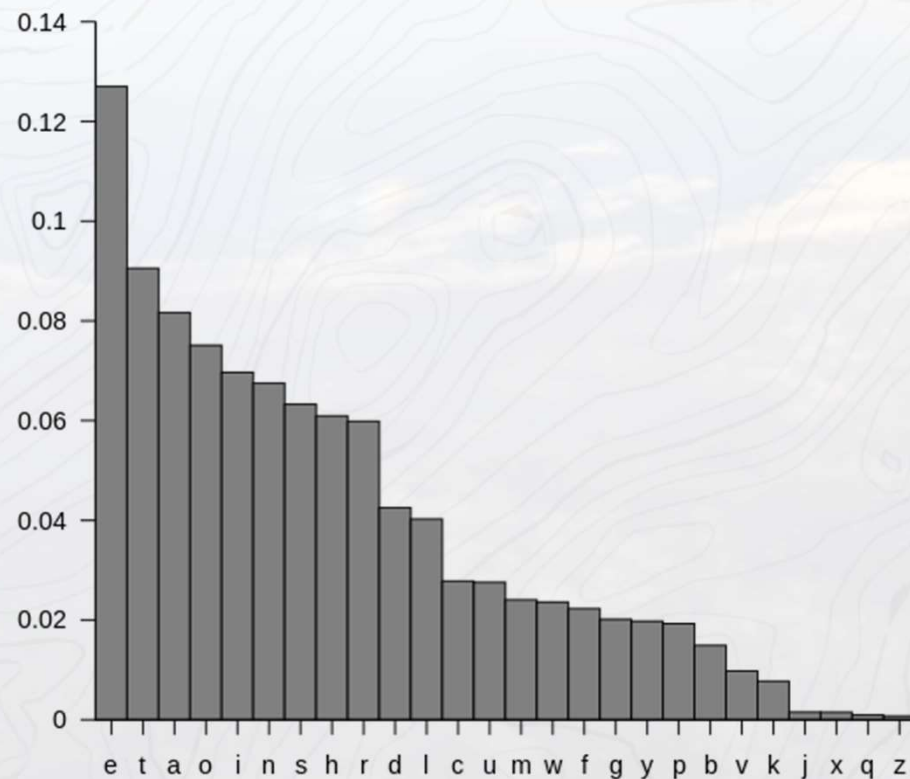
凯撒密码破解

暴力破解：位移一共有26种可能性，逐一尝试每种位移对应的明文，寻找可解释的结果。

语言学频率分析：

分析密文中每个字母出现的频率，然后将这一频率与正常情况下的该语言字母表中所有字母的出现频率做比较，估计出正确的偏移量。

有时还可以将频率分析从字母推广到单词，例如英语中，出现频率最高的单词是：**the, of, and, a, to, in...**。我们可以通过将最常见的单词的所有可能的25组密文，编组成字典，进行分析。



按大小排序后的英语字母频率



凯撒密码破解

示例：

密文：WKH TXLFN EURZQ IRA MXPSV RYHU WKH ODCB GRJ

频率出现最高的是字母R和字母H，推断这两个对应的应该是元音字母

WKH出现过两次，很有可能代表的是单词the。而且字母H也是高频率字母

猜测字母H代表的是字母E，偏移量为3

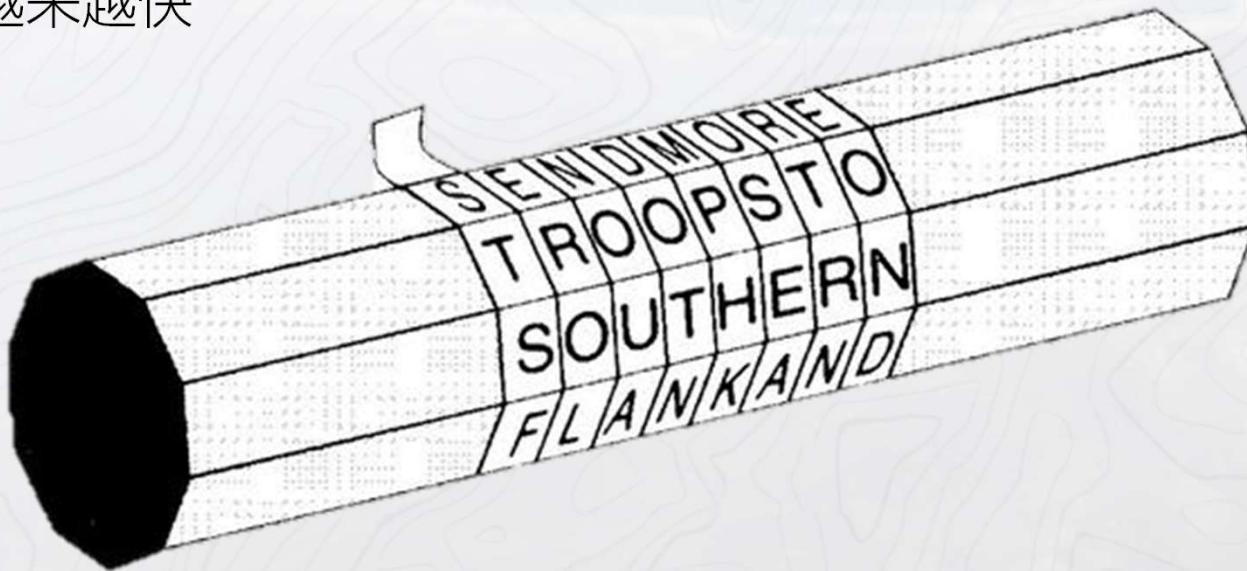
明文：THE QUICK BROWN FOX JUMPS OVER THE LAZY DOG

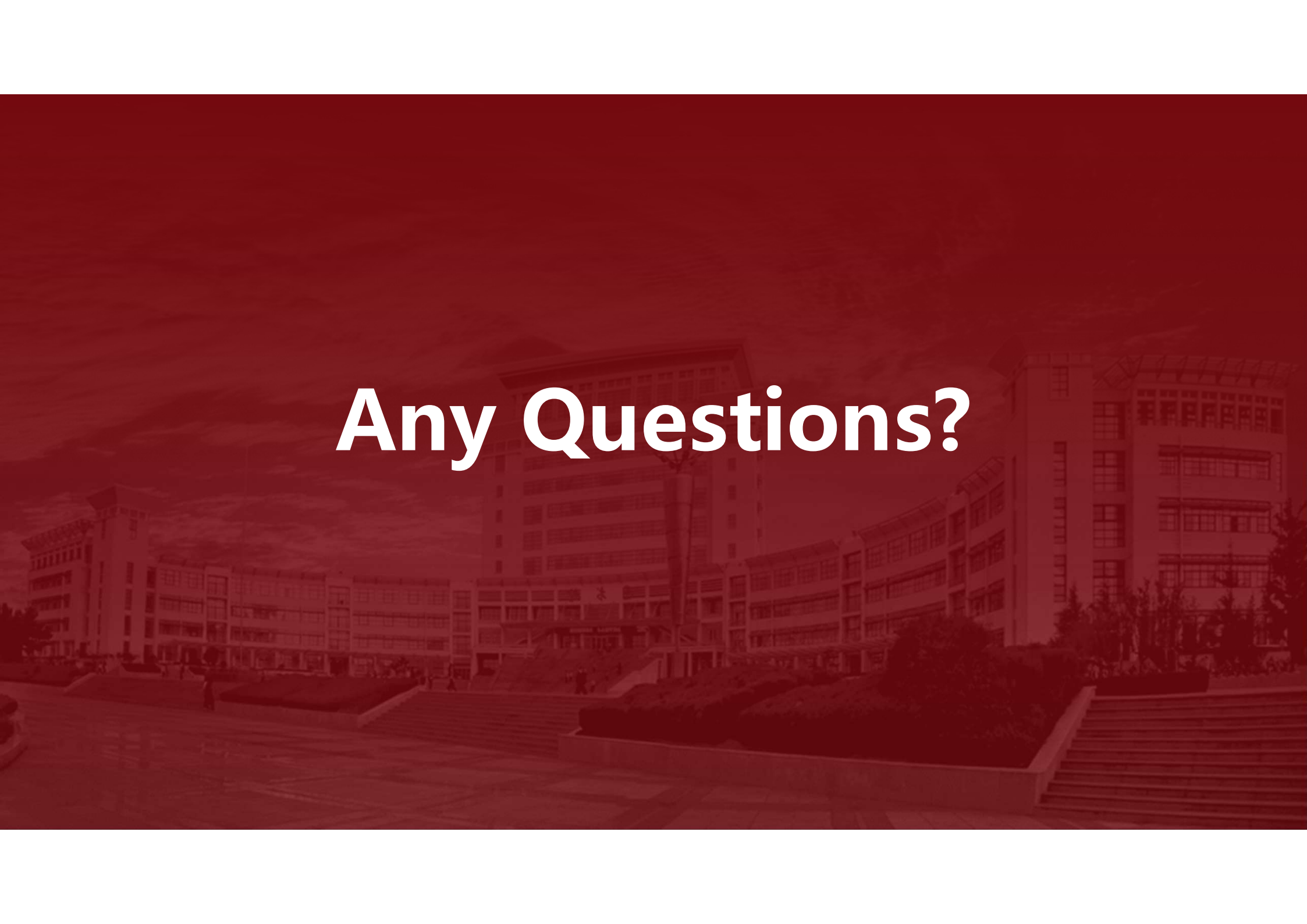
字符	空格	R	H	W	K	X	U	T	L	F	N	E	Z
次数	8	4	3	2	2	2	2	1	1	1	1	1	1
频率	19	9.3	7	4.7	4.7	4.7	4.7	2.3	2.3	2.3	2.3	2.3	2.3
Q	I	A	M	P	S	V	Y	O	D	C	B	G	J
1	1	1	1	1	1	1	1	1	1	1	1	1	1
2.3	2.3	2.3	2.3	2.3	2.3	2.3	2.3	2.3	2.3	2.3	2.3	2.3	2.3



凯撒密码破解

- 除了高频字母以外，低频字母也可能成为线索。
- 搞清楚开头与结尾能够成为线索，搞清单词之间的分隔也能够成为线索
- 密文越长越容易破解
- 同一个字母连续出现能够成为线索
- 破译速度会越来越快



A photograph of a university campus with several large, multi-story academic buildings and a central courtyard with stairs and landscaping. The entire image is overlaid with a semi-transparent dark red filter.

Any Questions?

维吉尼亚密码

爲天下儲人材 爲國家圖富強

—— 学无止境 气有浩然 ——



多表密码

不再是使用单一的数字进行位移加密，而是使用一组偏移量作为密钥
直观来说：由一些偏移量不同的恺撒密码组成

示例

正常字母表	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
替换字母表1（向右移动3位）	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C
替换字母表2（向右移动5位）	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E

密钥：1212，表示交替使用替换字母表1和替换字母表2来加密文本

明文：HELLO WORLD

密文：KJOQR BROWOI

解密时，接收方需要知道相同的密钥“1212”和使用的替换字母表。然后按照密钥的顺序使用相对应的字母表进行逆向替换，即使用替换字母表1时向左移动3位，使用替换字母表2时向左移动5位。



维吉尼亚密码

多表密码的一种特殊形式，通过使用一个关键字作为密钥，来决定每个字母在加密过程中应该使用哪一个替换字母表。

密钥中的每一个字母对应一个凯撒密码的偏移量，其中**A**代表**0**的偏移，**B**代表**1**的偏移，以此类推。加密时，将明文的每一个字母和密钥中相应位置的字母进行配对，使用配对的密钥字母所对应的凯撒密码对明文字母进行加密。密钥中的字母循环使用，直到所有的明文字母都被加密。

维吉尼亚密码相对于简单的替换密码更为安全，因为它不容易受到频率分析的攻击。频率分析是一种破解替换密码的方法，通过分析密文中字母的出现频率来推测明文。但由于维吉尼亚密码使用多个替换字母表，同一个字母在明文中的不同位置可能会被替换为不同的密文字母，这使得频率分析变得更加困难。



维吉尼亚密码

维吉尼亚密码

密钥: KEY

明文: HELLO WORLD

H位移K对应的偏移量----→ R

E位移E对应的偏移量----→ I

L位移Y对应的偏移量----→ J

L位移K对应的偏移量----→ V

O位移E对应的偏移量----→ S

.....

密文: RIJVS GSPVH

解密过程是加密过程的逆过程，接收方需要知道相同的密钥来确定每个字母的位移，并向相反的方向移动得到明文。

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
A	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
B	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A
C	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B
D	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C
E	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D
F	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E
G	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F
H	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G
I	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H
J	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I
K	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J
L	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K
M	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L
N	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M
O	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N
P	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
Q	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P
R	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q
S	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R
T	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S
U	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T
V	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U
W	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V
X	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W
Y	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X
Z	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y



维吉尼亚密码

维吉尼亚密码

加密练习:

明文: SECURE YOUR DATA

密钥: SAFE

解密练习:

密文: QEMWRRD IV SXCXRX

密钥: DATA

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
A	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
B	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A
C	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B
D	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C
E	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D
F	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E
G	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F
H	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G
I	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H
J	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I
K	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J
L	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K
M	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L
N	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M
O	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N
P	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
Q	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P
R	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q
S	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R
T	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S
U	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T
V	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U
W	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V
X	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W
Y	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X
Z	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y



维吉尼亚密码

维吉尼亚密码

加密练习:

明文: SECURE YOUR DATA

密钥: SAFE

密文: **KEHYJE DSMR IELA**

解密练习:

密文: QEMWRRD IV SXCXRX

密钥: DATA

明文: **NETWORK IS SECURE**

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
A	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
B	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A
C	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B
D	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C
E	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D
F	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E
G	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F
H	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G
I	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H
J	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I
K	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J
L	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K
M	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L
N	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M
O	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N
P	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
Q	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P
R	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q
S	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R
T	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S
U	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T
V	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U
W	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V
X	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W
Y	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X
Z	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y



维吉尼亚密码破解

1. 确定密钥长度
2. 确定密钥字相对位移
3. 穷举搜索密钥
4. 还原明文

密钥的长度的确定有两种方式，卡西斯基（Kasiski）检验和重合指数法。

Kasiski测试法的原理是：密文中出现两个字母相同的字组，他们所对应的明文字母相同的可能性很大，这样的两个密文字母组之间的距离可能是密钥长度的整数倍。

例：用SPILLTHEBEANS作为密钥加密“THE CAT IS OUT OF THE BAG”

明文	T	H	E	C	A	T	I	S	O	U	T	O	F	T	H	E	B	A	G
密钥	S	P	I	L	L	T	H	E	B	E	A	N	S	S	P	I	L	L	T
密文	L	W	M	N	L	M	P	W	P	Y	T	B	X	L	W	M	M	L	Z

间隔13

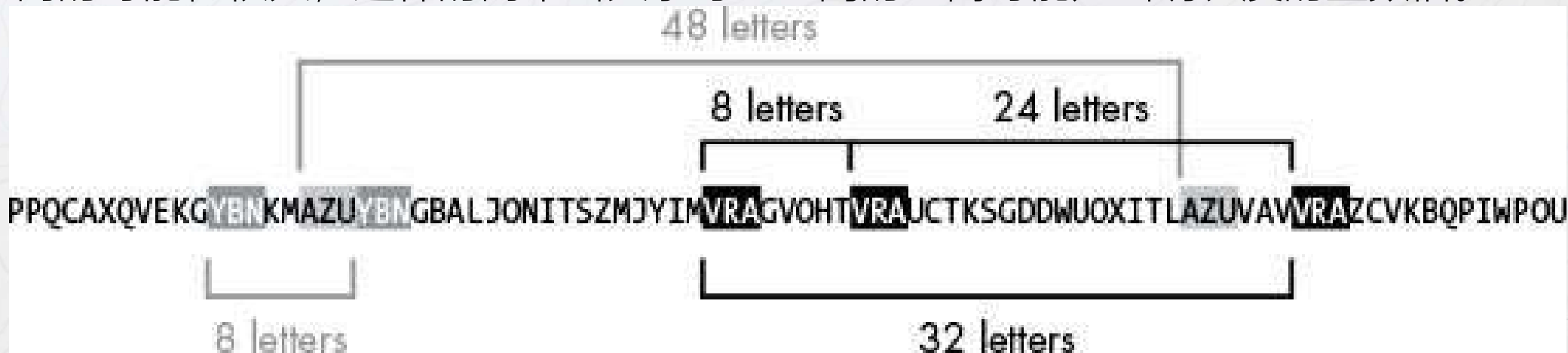


维吉尼亚密码破解

1. 确定密钥长度
2. 确定密钥字相对位移
3. 穷举搜索密钥
4. 还原明文

密钥的长度的确定有两种方式，卡西斯基（Kasiski）检验和重合指数法。

Kasiski测试法的原理是：密文中出现两个字母相同的字组，他们所对应的明文字母相同的可能性很大，这样的两个密文字母组之间的距离可能是密钥长度的整数倍。





维吉尼亚密码破解

重合指数是指字符串中两个随机元素相同的概率，记为 IC 。设某种语言由 n 个字母组成，每个字母 i 出现的概率为 $P_i (1 \leq i \leq n)$ ，则：

$$IC = \sum_{i=1}^n P_i^2$$

现实世界中密文有限，故从密文计算的重合指数总是不同于理论值，所以一般用 IC 的无偏估计值：

$$IC = \sum_{i=1}^n \frac{x_i(x_i - 1)}{L(L - 1)}$$

其中， x_i 是密文中第 i 个字母出现的次数， L 表示密文长度， n 表示字母数， $n = 26$ 。对于英文字符串，通过词频统计可以得到26个英文字母出现的期望概率。通过大量的英文样本，利用公式可得一般英文文本重合指数值约为0.65，而对于一个完全随机的字符串，其重合指数值约为0.038。



维吉尼亚密码破解

1. 初始化 $d = 1$ 。
2. 将密文分成 d 个子串，分别对每一个子串利用公式计算其重合指数，及它们的平均。若 d 为密钥长度，那么该平均值将接近0.065。
3. $d = d + 1$ ，重复步骤二。
4. 比较每一个 d 下的重合指数平均值，取最接近0.065的 d 值即为所求。

密钥长度	子串的重合指数					
1	0.041					
2	0.041	0.040				
3	0.040	0.043	0.042			
4	0.039	0.047	0.040	0.038		
5	0.055	0.059	0.073	0.060	0.066	
6	0.046	0.046	0.040	0.044	0.038	0.043
7						



维吉尼亚密码破解

确定长度后分组解密，每组都是一个凯撒密码

例：假设已经确定密钥长度是4，密文如下：

PPQCA XQVEKGYBNKMAZUYBNGBALJONITSZMJYIMVRAGVOHTVRAUCT
KSGDDWUOXITLAZUVAVVRAZCVKBQPIWPOU

第一组	PAEBABANZIAHAKDXAAAKIU	→进行频率分析→	A, I, N, W, X	W
第二组	PXKNZNLI MMGTUSWIZVZBW	→进行频率分析→	I, Z	I
第三组	QQGKUGJTJVVC GUTUVCQP	→进行频率分析→	C	C
第四组	CVYMYBOSYRORTDOLVRVPO	→进行频率分析→	K, N, R, V, Y	K

结果：

THOSEPOLICEOFFICERSOFFEREDHERARIDEHOMETHEYTELLTHEMAJOKETH
OSEBARBERSLENTHERALOTOFMONEY



维吉尼亚密码 in Python

```
CaesaTable = [[[]],[[]],...,[[]]] # 密码表
alphabet = 'abcdefghijklmnopqrstuvwxyz'

print("Please input your encryption key")
key = input('Key: ')
print("Please input your text")
text = input('Text: ')

pos = 0
ans = ""
```



维吉尼亚密码 in Python

```
for x in text:
    if x == " ":
        ans += " "
        continue
    i = alphabet.find(x)
    cipherKey = key[pos]
    j =
alphabet.find(cipherKey)
    ans += CaesaTable[i][j]
    if pos == len(key) - 1:
        pos = 0
    else:
        pos += 1
print("\nAns:", ans)
```

加密

```
for x in text:
    if x == " ":
        ans += " "
        continue
    cipherKey = key[pos]
    j = alphabet.find(cipherKey)
    i = [row[j] for row in
CaesaTable].index(x)
    ans += alphabet[i]
    if pos == len(key) - 1:
        pos = 0
    else:
        pos += 1
print("\nAns:", ans)
```

解密



练习

反转密码：使用一种将明文字符串反转的模式转换为密文。加密和解密的过程是一样的。要解密密文，用户只需将密文逆向即可得到明文。

例如：“meet me after the party” 得到的密文是ytrap eht retfa em teem

请编写对应的加密算法与解密算法程序并测试。



练习

反转密码进阶：对明文分组，再对每个分组单独反转得到分组的密文，然后连起来得到完整的密文。

例如：“meet me after the party” 分组为4，得到的密文是teem faem tret
apeh ytr

请编写对应的加密算法与解密算法程序并测试，暂时不处理空格，即去掉输入字符串中的所有空格。



练习

周期置换密码：根据指定周期先对明文分组，再根据指定的置换规则调整分组内字符的顺序得到分组密文，最后将所有分组密文合并得到完整的密文，分组后通常需要用指定字符填充不完整的组。

例如：“meet me after the party” 分组为4，置换规则为 $[1,2,3,4] \rightarrow [3,1,4,2]$ ，不完整的组用#填充。得到的密文是 **etme efma ettr epha t#ry**

请编写对应的加密算法与解密算法程序并测试，暂时不处理空格，即去掉输入字符串中的所有空格，分组后不满一组的用#填充。



练习

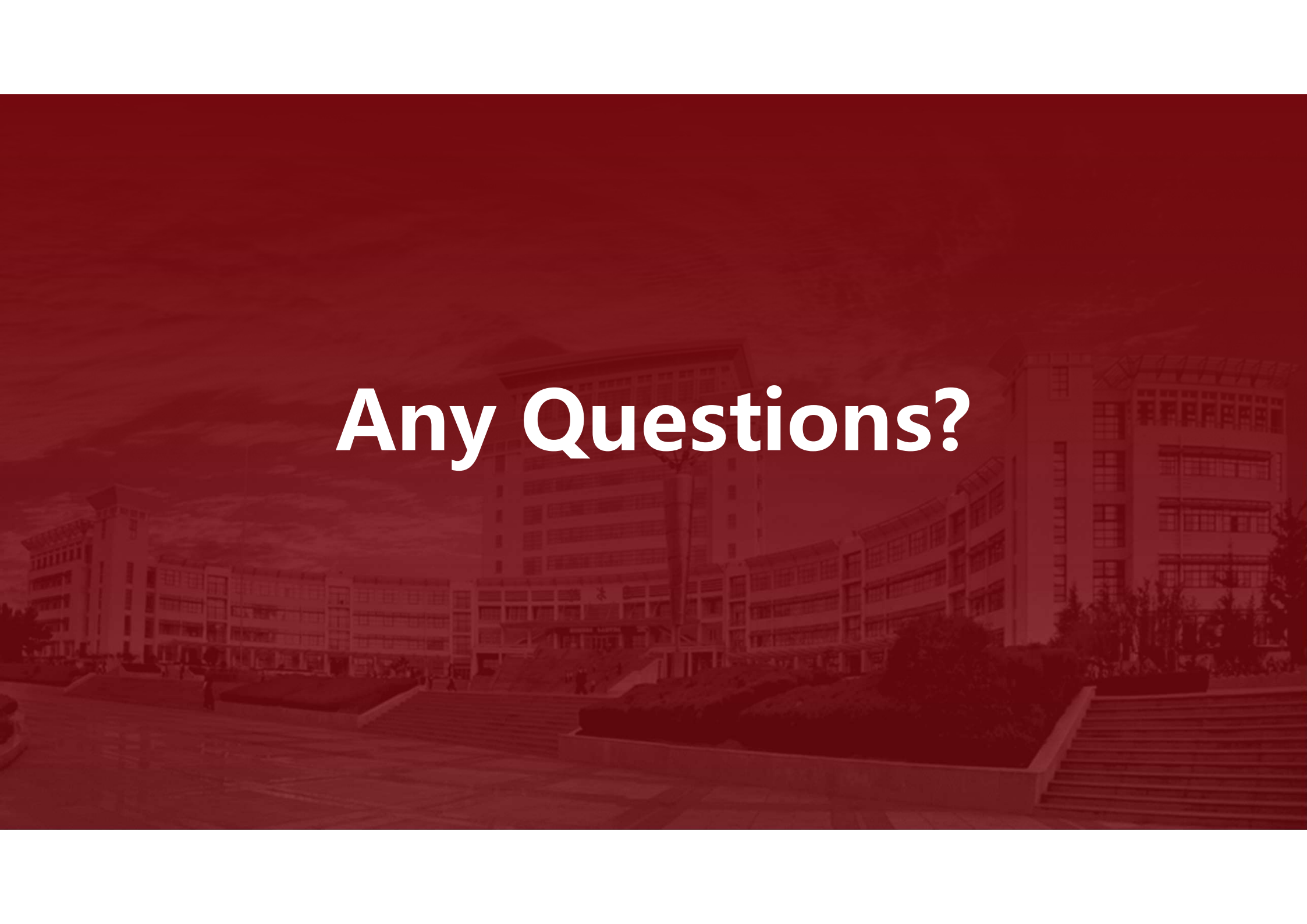
栅栏密码：按列写入，按行输出——把要加密的明文分成N个一组，然后把每组的第1个字连起来，形成一段无规律的话。

例如：用深度为2的栅栏技术加密明文“meet me after the party”

m	e	m	a	t	r	h	p	r	y
e	t	e	f	e	t	e	a	t	

得到的密文是mema tr hprie tef eteat

请编写对应的加密算法与解密算法程序并测试，暂时忽略空格的处理，即去掉输入字符串中的空格。

A photograph of a university campus with several large, multi-story academic buildings and a central courtyard with stairs and landscaping. The image is overlaid with a semi-transparent dark red filter.

Any Questions?